

Bubble Sort Algorithm in Java

AI Assistant

November 22, 2025

Abstract

This document provides a comprehensive overview of the Bubble Sort algorithm, a fundamental comparison-based sorting technique. It details the algorithm's working principle, presents a practical Java implementation, and thoroughly analyzes its time and space complexity. While simple to understand and implement, Bubble Sort's inefficiency for large datasets makes it primarily suitable for educational purposes or very small arrays.

Introduction

Bubble Sort is a straightforward sorting algorithm that repeatedly traverses a list, compares adjacent elements, and swaps them if they are in the wrong order. This iterative process continues until a full pass through the list completes without any swaps, indicating that the list is entirely sorted. Its simplicity makes it an excellent starting point for understanding sorting algorithms.

How Bubble Sort Works

The core mechanism of Bubble Sort involves comparing each pair of adjacent items within the array. If an element is found to be greater than its immediate successor, these two elements are swapped. This operation effectively "bubbles" the largest unsorted element to its correct final position at the end of the unsorted portion of the array during each complete pass. The algorithm repeats these passes until no swaps are performed in an entire iteration, which is the signal that the array is sorted.

Bubble Sort Implementation in Java

Below is a practical example demonstrating the implementation of the Bubble Sort algorithm in Java. This code includes an optimization where the sorting process terminates early if a pass completes without any swaps, indicating the array is already sorted.

```
public class BubbleSort {  
    /**  
     * Sorts an array of integers using the Bubble Sort algorithm.  
     * Includes an optimization to stop early if the array is already sorted.  
     * @param arr The array of integers to be sorted.  
     */  
    public static void bubbleSort(int[] arr) {  
        int n = arr.length;  
        for (int i = 0; i < n - 1; i++) {  
            boolean swapped = false; // Flag to detect if any swaps occurred in this pass  
  
            // Last i elements are already in place, so we don't need to check them  
            for (int j = 0; j < n - i - 1; j++) {  
                if (arr[j] > arr[j + 1]) {
```

```

        // Swap arr[j+1] and arr[j]
        int temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
        swapped = true; // Mark that a swap occurred
    }
}

// If no two elements were swapped by inner loop, then array is sorted
if (!swapped) {
    break;
}
}
}

public static void main(String[] args) {
    int[] arr = {64, 34, 25, 12, 22, 11, 90};

    System.out.println("Original array:");
    for (int i = 0; i < arr.length; i++) {
        System.out.print(arr[i] + " ");
    }
    System.out.println("\n"); // Add a newline for better readability

    bubbleSort(arr); // Call the sorting method

    System.out.println("Sorted array:");
    for (int i = 0; i < arr.length; i++) {
        System.out.print(arr[i] + " ");
    }
    System.out.println(); // Add a newline at the end
}
}

```

Complexity Analysis

Understanding the efficiency of an algorithm is paramount. Here, we analyze the time and space complexity of Bubble Sort.

Time Complexity

The time complexity describes how the running time of an algorithm scales with the input size (n).

- **Worst and Average Case Complexity: $O(n^2)$**

This scenario typically occurs when the array elements are sorted in reverse order or are randomly arranged. In these cases, the algorithm performs approximately $n(n-1)/2$ comparisons and swaps, leading to quadratic growth in execution time.

- **Best Case Complexity: $O(n)$**

The best case arises when the array is already sorted. With the optimization implemented (checking for swaps in each pass), the algorithm can detect a sorted array in a single pass, resulting in linear time complexity.

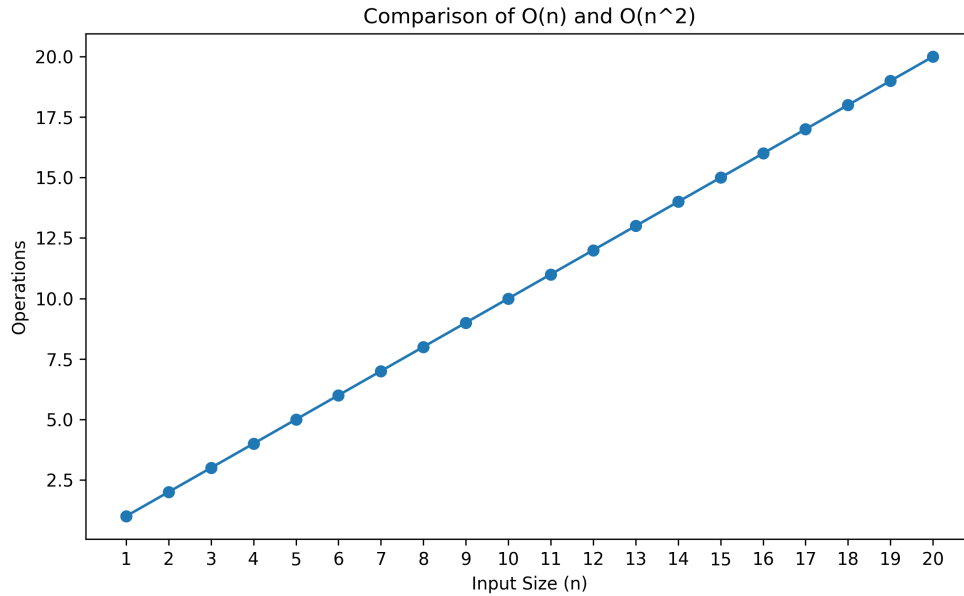


Figure 1: Comparison of $O(n)$ and $O(n^2)$ Time Complexity

Space Complexity

Space complexity refers to the amount of auxiliary memory an algorithm requires to execute.

- **Space Complexity: $O(1)$**

Bubble Sort is an in-place sorting algorithm. It only requires a constant amount of extra memory for temporary variables (e.g., the `temp` variable used during swaps), irrespective of the input array's size. This makes it very memory efficient.

Conclusion

Bubble Sort is a fundamental algorithm, valued for its conceptual simplicity and ease of implementation, making it an excellent tool for educational purposes. However, its quadratic time complexity, $O(n^2)$, renders it highly inefficient for sorting large datasets. For practical applications demanding high performance, more advanced algorithms such as Merge Sort, Quick Sort, or Heap Sort are significantly more suitable. Bubble Sort's utility is primarily confined to academic settings or scenarios involving very small arrays where simplicity outweighs performance concerns.